

Progetto DDR Perimeter & Radar | [LINK AL PROGETTO](#)

(https://github.com/savino1703/issLab2026/tree/main/robot_ddr).

Requisiti

Costruire un sistema software che comanda un Differential Drive Robot (DDR) in modo che, partendo dalla posizione iniziale HOME, il robot si sposti lungo il perimetro di una stanza rettangolare vuota. Il DDR si deve fermare quando tornato in HOME.

Il DDR è anche sensibile ai dati rilevati da un Radar. Quando il Radar rileva un 'intruso' a una distanza minore di una distanza prefissata **DMIN**:

- Se il DDR si sta muovendo, il DDR **si ferma** fino a che il Radar non rileva più un 'intruso' così vicino.
- Se il DDR è fermo in HOME, il DDR **ruota su se stesso** fino a che un 'intruso' così vicino scompare.

Analisi dei Requisiti

Dalla lettura attenta del testo emergono le seguenti entità logiche e concetti fondamentali da chiarire:

- **Differential Drive Robot (DDR):** Un oggetto fisico già a disposizione del committente, non un punto, il quale occuperà una certa area, per semplicità quadrata di lato R. Il DDR, comandato dal sistema, può muoversi in avanti e ruotare su sè stesso di 90°, non è altro che un semplice attuatore di comandi.
- **Stanza rettangolare vuota:** Un rettangolo in un piano euclideo con base B e altezza H espresse in metri, indicando con coordinate X e Y una posizione all'interno della tale. Essa può anche essere vista come una griglia le cui celle sono misurate in unità robotiche
- **HOME:** È un'area, una posizione iniziale in cui il DDR si trova all'interno della stanza, nonché posizione finale del percorso. Questa non è altro che una cella all'interno della griglia.

[#

```
data class Posizione(val x: Int, val y: Int)
```

```
data class Stanza(val B: Double, val H: Double, val R: Double) {
```

```
val larghezza: Double = B * R
```

```
val altezza: Double = H * R
```

```
val home: Posizione = Posizione(0, 0)
```

```
}
```

```
#]
```

- **Radar:** Un'entità (fisica o simulata), un attore, che controlla, a intervalli regolari, la distanza tra il DDR ed eventuali 'intrusi' (ostacoli imprevisti posizionati lungo il perimetro) ad una distanza inferiore di **DMIN**, generando poi eventi asincroni utili come informazioni per il sistema, ovvero la distanza rilevata D.
- **DMIN:** Una soglia numerica predefinita. Serve come trigger per scatenare un comportamento anomalo del robot, quando rilevato dal Radar.

esistono due metodi per fare comunicare il radar con il sistema:

Il radar comunica con eventi la distanza rilevata degli intrusi

```
System ddr
```

```
Event radar : distance(D)
```

```
Context ctxddr ip [ host="localhost" port=8010 ]
```

```
QActor radar context ctxddr {
```

```
[#
```

```
var D=0
```

```
var Timer=2200
```

```
#]
```

```
State s0 initial{
```

```
println("starting")
```

```
}
```

```
Goto radarDistanza
```

```
radarDistanza{
```

```
emit radar : distance(D)
```

```
}
```

```
Transition t0
```

```
whenTimeVar Timer -> radarDistanza
```

```
}
```

Il radar comunica la distanza in risposta a richieste del sistema

```
System ddr
```

```
Request getDistance : getDistance(D)
Reply radar : distance(D) for getDistance
Context ctxddr ip [ host="localhost" port=8010 ]
```

```
QActor radar context ctxddr{
[#
var D = 0
var Timer = 2200
#]
State s0 initial {
println("starting")
}
Goto radarDistanza

State radarDistanza{
//
}
Transition t0
whenTimeVar Timer -> radarDistanza
whenRequest getDistance -> handleRequest

State handleRequest{
onMsg( getDistance : getDistance(D) ){
replyTo getDistance with radar : distance( $D )
}
}
Transition t0
whenTimeVar Timer -> radarDistanza
}
```

Comportamento Ibrido del Robot

Il sistema richiesto non è un semplice esecutore di comandi sequenziali. Il DDR, in contemporanea, deve poter:

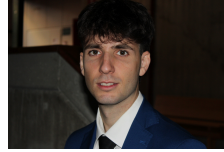
1. Esplorare il perimetro in modo autonomo e terminare il lavoro.
2. Interrompere istantaneamente il suo task se l'ambiente esterno cambia (Radar rileva `distanza < DMIN`).

Per questi motivi necessita di due stati distinti:

1. Uno **stato in movimento** in cui segue il perimetro

2. Uno **stato in cui è fermo** e rotea finchè rileva intrusi nelle vicinanze (questo stato sarà sia quello iniziale che quello finale).

By Paolo Savino email: paolo.savino2@studio.unibo.it



GIT

repo: github.com/savino1703/issLab2026
(<https://github.com/savino1703/issLab2026>).